

Digital Lab: Sampling, Fourier Transforms, Mixers, and Down-Converters

By Patrick Kantorski

March 7, 2014

Abstract

This report was written for a radio astronomy undergraduate course at UC Berkeley. For this lab, my partners and I examined and tested various concepts involved with digital signal processing. From Nyquist's criterion to signal recovery, we performed experiments and collected data. Additionally, digital and analog mixing (along with SSB and DSB mixing) was examined. We also provided a design for our FIR filter and test it by using a digital down converter.

1 Introduction

Analog signals can be converted to digital signals, but only at a specified sample rate and for a specified period of time. Many experiments in Radio Astronomy require recording and extracting information from radio signals. Although this is almost always done digitally, some parts of the signal mixing and filtering process can be performed with analog components. Signals possess many important properties and can even be mixed together and filtered out.

For this lab, my partners and I examined these concepts by using and implementing digital signal processing with a CPU based computer, function generators, ZAD-1 signal mixer, and an FPGA based computer called the ROACH. Plots and experimental results derived required extensive coding performed through the programming language Python. From Nyquist's criterion, signal sampling can be seen to have inherit limitations which must be considered to properly sample data. Additionally, we explored both analog and digital methods of signal mixing and compared the advantages and disadvantages of each. Techniques like as Fourier filtering can be used on signals in order to recover specific frequencies. Toward the end of this lab, my partners and I examined and programmed a FIR filter using the ROACH similar to that of a bandpass.

2 Materials & Methods

2.1 Nyquist's Criterion

Digital data collection functions by collecting sets of data points at a particular sample rate or sample frequency. One of the fundamental underlying concepts involved in digital data sampling is Nyquist's criterion. Nyquist's criterion for digital data collection states that there is a minimum sample frequency required to digitally sample a waveform in facsimile. To reproduce a sampled waveform, at least two data points must be taken per period. The Nyquist frequency is shown in Equation 1.

$$v_{sample} = 2 \times v_{signal} \tag{1}$$

Because sampled points are separated in time, oscillations in the signal that occur between two data points will not be recorded. If Nyquist's criterion is not satisfied during data collection, aliasing will inevitably occur. Aliasing is when a sampled waveform facsimiles a lower frequency than what is being measured. An example of aliasing is shown in Figure 1.

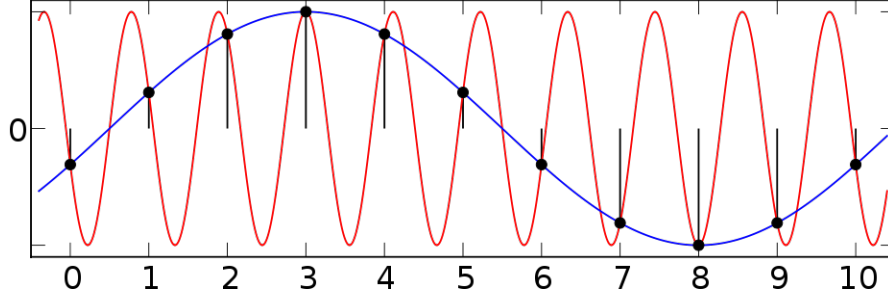


Figure 1: Although a clear sine wave is produced from sampling the signal shown above in red, there are several points missed in-between each of the data points. This misrepresentation of the original signal is called aliasing.

To demonstrate Nyquist's criterion and aliasing, my partners and I used the Pulsar sampler card, a computer, and a SRS Function Generator to take digital waveform data samples of sine waves generated at different signal frequencies. Additionally, we set our sampling rate to $v_{sample} = 1MHz$. Because the Nyquist frequency, following Equation 1, was precisely $v_{signal} = 0.5MHz$, for observational purposes, we took data at signals ranging from $v_{signal} = 0.1MHz$ to $0.9MHz$. The sampled waveforms for these signals are shown in Figures 2 and 3.

2.2 Fourier Analysis

A Fourier transform is a mathematical transformation used to transform signals between the time or spatial domain and frequency domain. In signal and data processing, Fourier transforms can be used to generate a power spectrum of a digital signal. Besides application, Fourier transforms are also very useful because sine waves are quite easy to work with mathematically and form a basis over all function spaces. Fourier transforms also distinguish negative and positive frequencies in a spectrum. A negative frequency is the same as a positive frequency except that instead of the cosine leading the sine by $\frac{\pi}{2}$, it trails by $\frac{\pi}{2}$ (see Equation 2). For functions with real-values, positive and negative frequencies contain the same information. In complex-valued functions, positive and negative frequencies contain independent information, making the transformation not centered about the origin.

$$e^{i\omega t} = \cos(\omega t) + i \sin(\omega t) \quad (2)$$

$$E(v) = \frac{1}{T} \int_{-T/2}^{T/2} E(t) e^{2\pi j v t} dt, \quad E(t) = \frac{1}{F} \int_{-F/2}^{F/2} E(v) e^{-2\pi j v t} dt \quad (3)$$

Fourier transforms are also invertible, allowing one to go from either go from the time to the frequency domain or from the frequency domain back to the time domain using the inverse transformation. In the proper analytic formulation (see Equation 3), the boundaries of integration

for the forward and inverse FT are from negative to positive infinity. However in application, these boundaries cannot be infinite.

Two types of Fourier transforms were implemented and examined in this lab. One of them is known as the Discrete Fourier Transform or DFT. A DFT converts a finite list of equally spaced data samples taken from a function or signal into the list of coefficients of a finite combination of complex sine waves that has those same sample values, ordered by their frequencies. Since DFTs function with a finite amount of data, they can be implemented in computers by numerical algorithms or even dedicated hardware. Using the programming language Python, a DFT was constructed by my partners and I to view the power spectrum of the Nyquist sampling data.

DFTs have N input numbers and N output numbers, making the number of calculations proportional to N^2 . However, for larger N values, the calculation can take a long time. Because of this, DFTs are inherently limited. To get around this problem, the Fast Fourier Transform, also known as the FFT, was developed as to perform Fourier transforms a lot quicker. Unlike a DFT, in a FFT, the number of calculations is proportional to $N \log_2 N$. Since this makes larger N value transforms more practical, FFTs are typically used more often than DFTs.

2.3 Signal Mixing

While working with radio transmissions, desired signals are often at frequencies higher than what digital sampling tools are capable of handling. By using a heterodyne mixer, radio signals can be shifted down to a lower frequency so that they can be properly sampled. A local oscillator converts an incoming signal to a different frequency by mixing the signal with a local oscillated (or LO) signal. When two signals are mixed, their values are multiplied in the time domain. Additionally, this is equivalent to taking the convolution of the signals in the frequency domain (see Equation 4).

$$f(t) * g(t) = \int f(\omega)g(\omega)e^{i\omega t}d\omega \quad (4)$$

The convolution is a mathematical operation used in applications ranging from photo editing to radio astronomy. In simple terms, taking the convolution of two signals effectively “blurs” or smears one over the other. In this lab, my partners and I examined both analog and digital mixing.

To analog mixing our signals, we used a Mini-Circuits ZAD-1 and chose a specific LO frequency to mix along with another sine wave set at a different frequency (see Equation 5). The ZAD-1 is a balanced mixer with identical “R” and “L” ports. It also does not couple to DC or very low frequencies. The “I” port is coupled differently and will handle voltages all the way down to and including DC. The mixing process functions no matter which two ports are used as inputs. We also had to make sure that we had enough samples per period to generate a reasonable facsimile of the sine wave. This forced us to sample above at least twice the Nyquist frequency and set the number of points sampled to be large enough to give produce at least a few periods of the “slowest” sine wave.

$$v_{signal} = v_{lo} \pm \delta v \quad (5)$$

A single sideband mixer, or SSB, requires mixing with pure sine waves which have both a real and imaginary part. Because of this, SSB mixing removes half of the sidebands created. A double sideband mixer, or DSB, splits the signal into a lower sideband and an upper sideband. We first took our data samples at the mixed signal frequency $v_{lo} + \delta v$. and $v_{lo} - \delta v$. Since both signals were by the SRS Function Generator, they were inherently real and consisted of both $e^{i\omega t}$ and $e^{-i\omega t}$

components (see Equation 2). This made the mixed output of our analog mixed signal composed of four different frequencies; $v_{lo} + \delta v$, $v_{lo} - \delta v$, $-v_{lo} + \delta v$, and $-v_{lo} - \delta v$.

2.4 Using the ROACH

The Reconfigurable Open Architecture for Computing Hardware (or ROACH) is an open-source board developed at the University of California, Berkeley by Dan Werthimer and Aaron Parsons from the CASPER collaboration. The circuit board of the ROACH used consists of a Virtex-5 FPGA made by Xilinx along with a CPU. The CPU allowing users to SSH directly to the board using a special linux kernel designed by CASPER. This kernel also allows for automatic interface support between programming the FPGA.

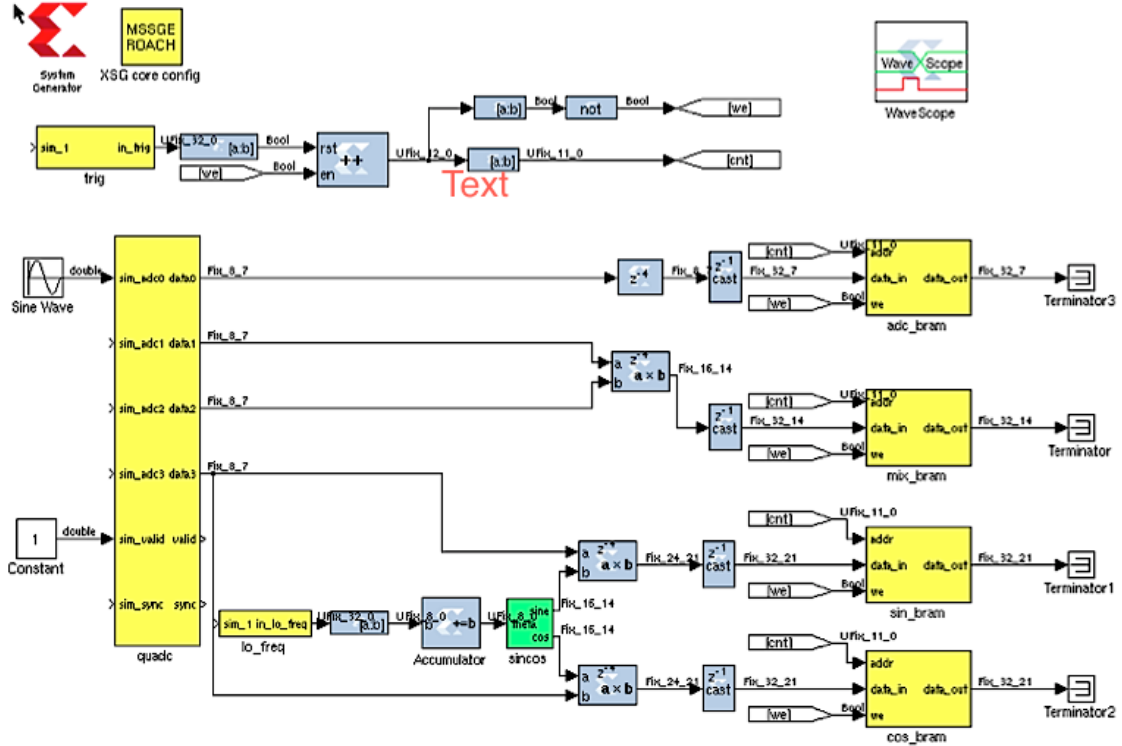


Figure 2: The picture above shows the data sampler program design used on the ROACH, written in the programming language Simulink. This design compiles data into the “adc snaps” BOF files which are used during digital mixing and sampling.

The ROACH is system constructed from a computer running a Linux kernel and a Field Programmable Gate Array (or FPGA) processor. Unlike CPU processors, FPGAs can be configured to perform multiple tasks simultaneously by a series of programmable logic gates. In this lab, our FPGA processor was synchronized by a 200MHz clock. By connecting remotely, my partners and I programed the ROACH with configurations set to take data samples through altering the FPGA’s registers. Data taken from the ADC output is stored in the form of block random access memory (or BRAM). After being written, data then can be copied from the ROACH for speculation and analysis.

2.5 FIR Filtering

The Finite Impulse Response filter (or FIR) works by taking the convolution of two signals in the time domain. Fourier filtering works well with filtering signals after they have been sampled, but not so much with signals in real-time. However, the digital implementation can still be achieved through programming a FPGA. By taking the Fourier transform of a signal, multiplying (or convolving) the FFTs by a filter, and then inverting transform to get the filtered signal, real-time digital filtration can be achieved (see Figure 3).

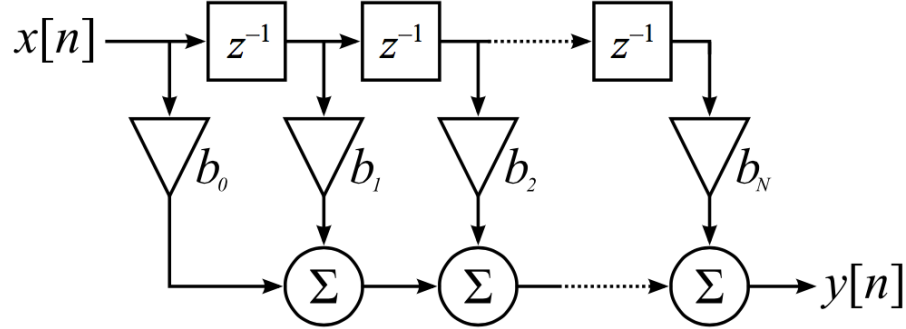


Figure 3: Figure above is a schematic diagram of how a FIR filter works. Coefficients or “taps” are represent by b_N .

Real-time digital filters perform a convolution on samples received as the signal is sampled. The implementation of an FIR filter on the FPGA allows us to convolve an incoming signal with other waveforms in the time domain. Because the filter my partners and I used has 8 taps, we decided to choose 8 real and 8 imaginary coefficients for our filter. We did this by writing the coefficient registers to the FPGA. As the signal is sampled, the FPGA multiplies about 12 of the last eight points from the ADC with their corresponding coefficients and then sums them. As the clock cycles, the next point sampled is fed in and the process repeats. The values are then summed up to form a new “convolved” waveform. Since the sums are largest when the signal falls along the filter, the FIR filters out portions of the signal that are similar to the filter’s shape and zeros out the rest.

3 Observations & Data Analysis

3.1 Digital Data Sampling

To demonstrate aliasing and Nyquist’s criterion, data samples were collected from a SRS Function Generator, over the command-line in Python. The number of samples collected was 256 with a sample frequency of 1MHz while the voltage was set to precisely $1V_{pp}$.

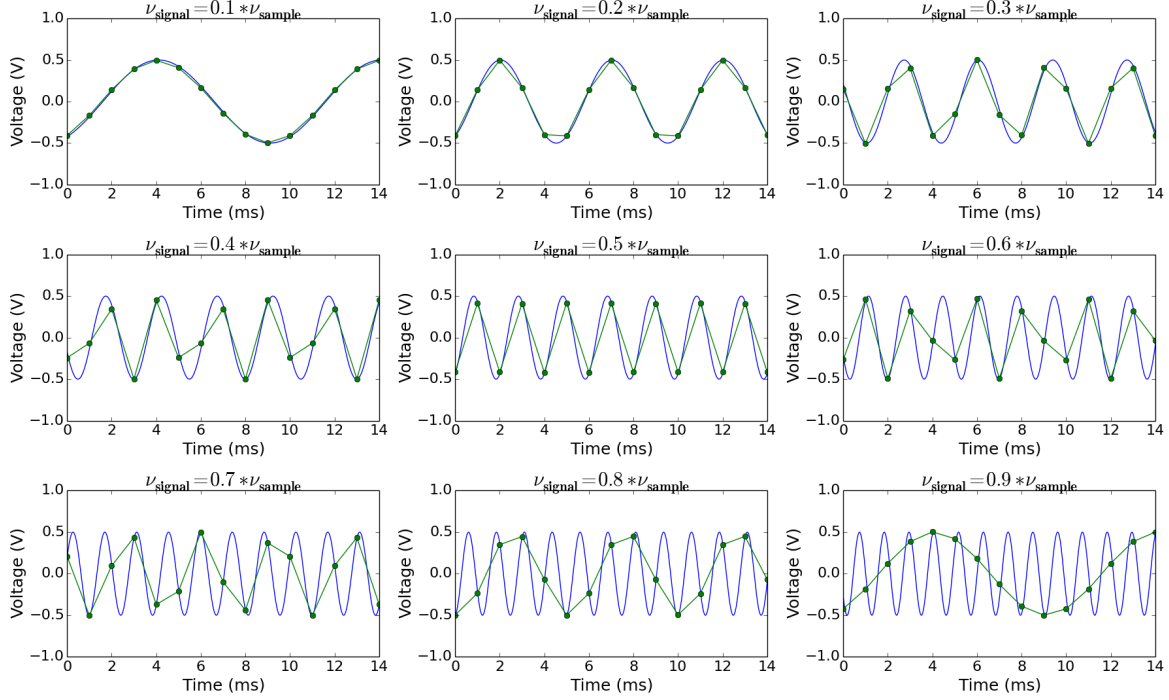


Figure 4: The graph above shows data samples of 1MHz, $1V_{pp}$ sine wave collected through the command-line using Python and a local oscillator and a Pulsar sampler.

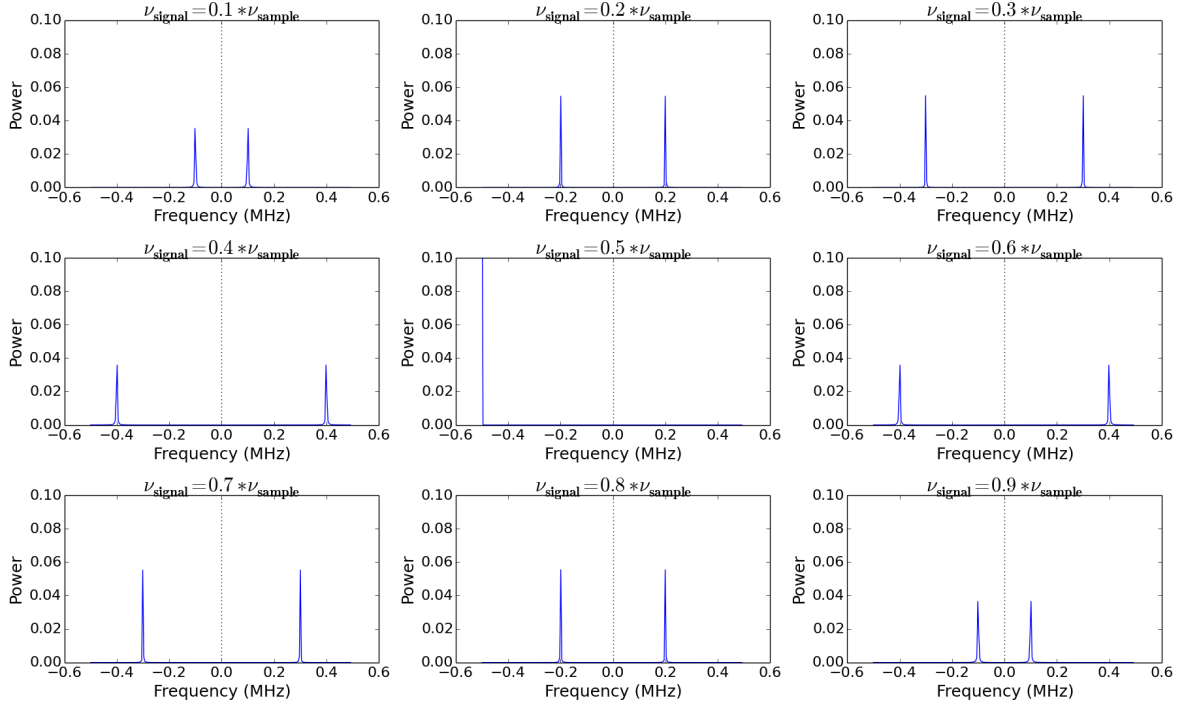


Figure 5: The graph above shows the DFT of the data samples from Figure 3.

As shown in Figure 4, the signal viewed ranged from $v_{signal} = 0.1MHz$ to $0.9MHz$. For signals above at the Nyquist frequency $v_{signal} = 0.5MHz$, the sampled waveform does not have any rising or lowering fluctuations between the high and low points on the graph. Additionally, the voltage values at the Nyquist frequency are consistently at about the same value and same for both high and low values. This is because the incoming signal is sampled at twice per period. Because there is only are only high and low data points, the shape of the waveform is triangular as opposed to a clear facsimile of the sinusoid. Nevertheless, the $0.5MHz$ frequency is still apparent in both its time domain representation and Fourier transformation (see Figure 5).

When the signal frequency is above $0.5MHz$, the effects of aliasing becomes distinctive. After falling below the Nyquist frequency, the data plots begin to revert. At the signal frequency of $0.9MHz$, a clear sine wave can be viewed, similar to the original $0.1MHz$ signal sample. Also, when the signal frequency is exactly equal to the sample frequency, every data point taken will fall on the same position, forming a line.

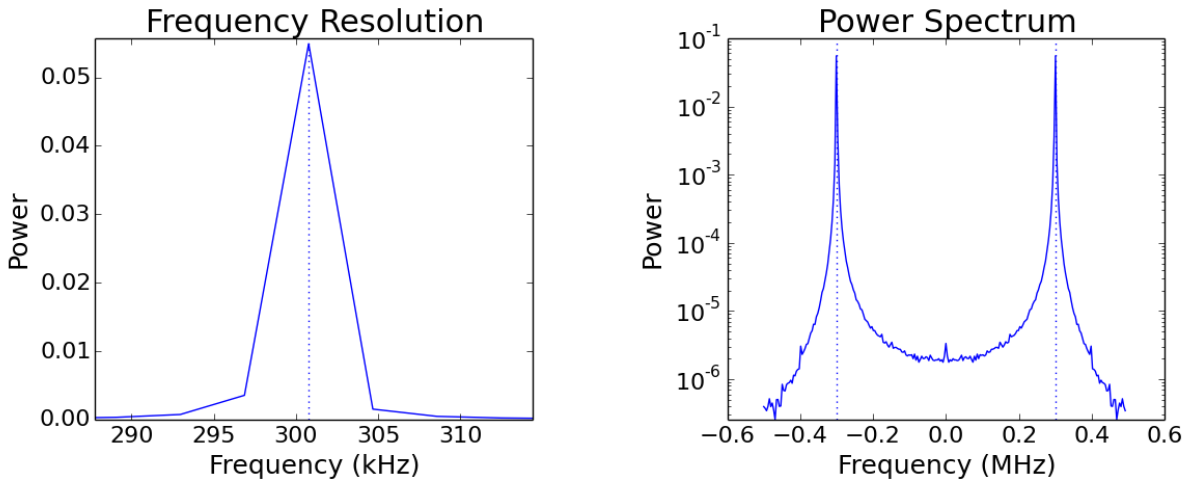


Figure 6: The graph above shows the frequency resolution and power leakage from the $0.3MHz$ data sample used in Figures 1 and 2. The frequency resolution is half of the width of the spike.

Equation 6 represents the frequency resolution of a sampled waveform. Frequency resolution can be thought of as the width between each frequency in the power spectrum. For the power spectrum shown in Figure 4 and 5, the frequency resolution is approximately $\Delta v \approx 3.906kHz$.

$$\Delta v = \frac{1}{T_{sample}} = \frac{v_{sample}}{N} \quad (6)$$

Spectral leakage for the $0.3MHz$ data sample are shown in Figure 6. The power spectrum is the average distribution of the power of a signal represented as a set of components harmonically related to the fundamental frequency, or the lowest frequency of the signal. When all the power of the signal is at frequencies that are integer multiples of the fundamental frequency, it can then adequately be represented by the Fourier transformation. However, a problem arises when there is power in the signal at frequency components that are not harmonics of the fundamental. When this happens, the power of these components becomes misrepresented. The term spectral leakage refers to this misrepresentation of components that are not integer multiples of the fundamental frequency.

3.2 Analog Mixing

Using two SRS synthesizer oscillators, my partners and I DSB mixed a 100kHz, -7dB_m sine wave with a separate 105kHz and 95kHz, -3dB_m signal using a Mini-Circuits ZAD-1. Using the Pulsar card, we chose sample at a rate of 4MHz and collect $N = 16000$ data points, making $T_{\text{sample}} \approx 4\text{ms}$. The resulting mixed signal is shown in Figure 7 and power spectrum in Figure 8.

v_{lo}	v_{signal}	$v_{lo} \pm v_{\text{signal}}$	$-v_{lo} \pm v_{\text{signal}}$
100kHz	105kHz	205kHz, -5kHz	5kHz, -205kHz
100kHz	95kHz	195kHz, 5kHz	-5kHz, 195kHz

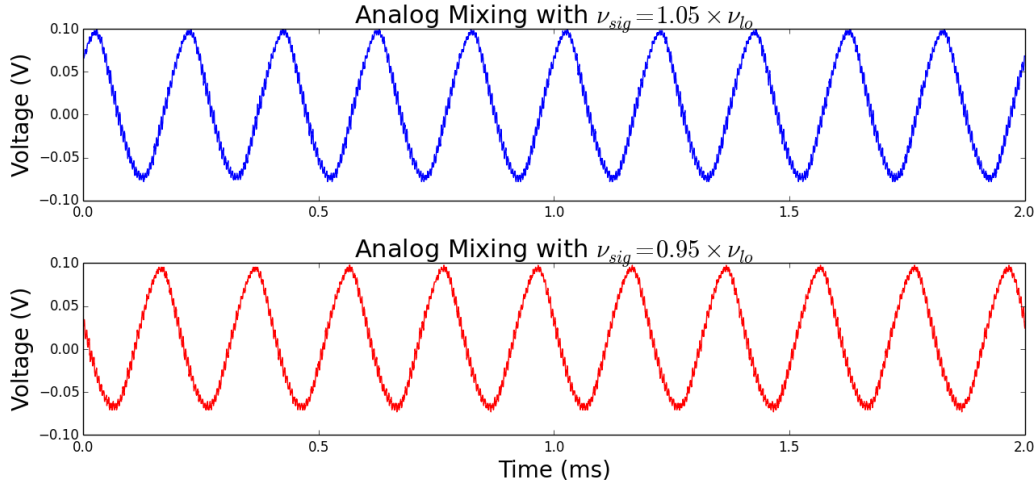


Figure 7: This graph shows an analog DSB signal mixed between a 100kHz and 105kHz signal obtained through Python with $v_{\text{signal}} = 1.05v_{lo}$ on the left and $v_{\text{signal}} = 0.95v_{lo}$ on the right. A sample size of 16000 data points were sampled at a sample frequency of 4MHz using the Pulsar card. There is also an apparent DC offset of about 0.01V.

As seen on Figure 7 and the table above, the mixed signal has four distinct positive and negative frequencies. Additionally, since the $\pm 5\text{kHz}$ signal is degenerate, there are only six unique frequencies. For the positive v_{lo} frequencies, the output splits into a lower sideband $v_{lo} + v_{\text{signal}}$ and an upper sideband $v_{lo} - v_{\text{signal}}$, forming the double sideband (or DSB) mixer. Single sideband (or SSB) mixers require the mixing of pure sine waves that have both a real and imaginary part. With complex sine waves, we are able to eliminate one of the two sidebands. The upper and lower sideband of the mixed signal are 205kHz and 5kHz signals respectively. The upper and lower sideband are symmetric for the both negative and positive v_{lo} .

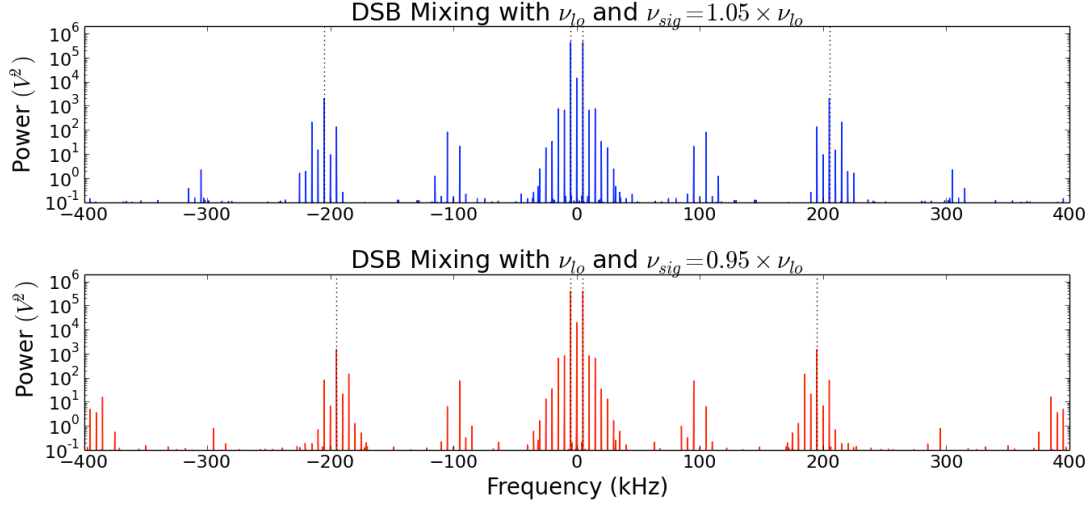


Figure 8: The graph above shows the DSB mixing power leakage spectrum of the mixed wave form in Figure 6. Dotted-line is placed along the distinctive frequencies of the mixed signal.

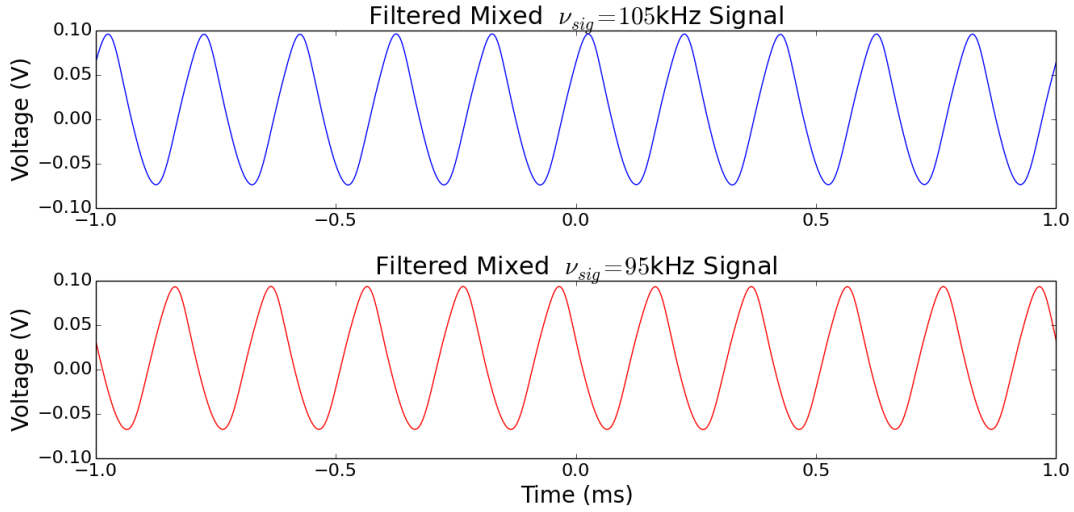


Figure 9: The graph above shows the inverse Fourier transform or Fourier filter of the DSB mixing signal from Figure 6, isolating the 5kHz frequency.

Since the upper sideband distorts the 5kHz signal, I decided to filter it out. To do this, using Python, I made the values of each frequency higher on the FFT of the signal zero and then took the inverse FFT (or IFFT) to retrieve filtered signal. This was done in Python by first isolating 5kHz then using the “IFFT” feature built into the package “NumPy.” This method of digital filtration is called Fourier filtering and can be seen in Figure 9. The band of a Fourier filter is somewhat arbitrary and really depends on some particular range of interest. In the data, we chose also prevented any mixing products that may have occurred as a result of any DC bias mixed into our signals. After isolating the 5kHz and taking the inverse Fourier transform of the power spectrum, the filtered signal was be retrieved from the mix.

3.3 Digital Mixing (DSB and SSB) on an FPGA

3.3.1 DSB Mixing

Digital DSB mixing is similar to analog DSB mixing because of how the signals are mixed. However, instead of mixing signals with physical, analog components, the mixing is performed digitally in a computer. But like analog mixing, there are still some inherent limitations. Even if the two signals being mixed together do not violate the Nyquist criteria individually, if the upper sideband of the signal is higher than the Nyquist frequency, then aliasing will still occur. Furthermore, if a DC offset is present in any of the separate signals, then it will also show up as spikes on the power spectrum of the mixing product around their frequencies.

v_{lo}	v_{signal}	$v_{lo} \pm v_{signal}$	$-v_{lo} \pm v_{signal}$
1MHz	1.05MHz	2.05MHz, -50kHz	50kHz, -2.05MHz
1MHz	950kHz	1.95MHz, 50kHz	-50kHz, -1.95MHz

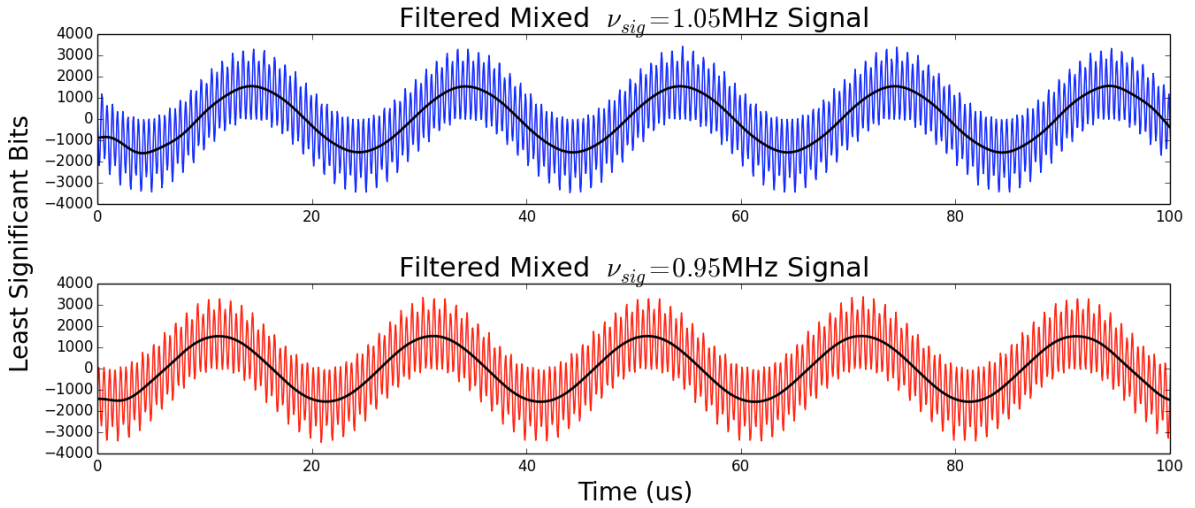


Figure 10: The graph above shows a DSB mixed signal taken from the ROACH using a LO frequency of about 100MHz and a signal frequency of about 95MHz. The inverse Fourier transform of the signal is mapped on top of the mixed signal, isolating 50kHz. Data was collected under a sample rate of 200MHz.

As we have shown before, mixing of signals v_{lo} and v_{signal} is a fairly simple process. Digital mixing of two signals was performed using the ROACH with our output results written to a BRAM file. Following the same signal mixing procedures as Section 3.2 should produce similar results when using the ROACH to digitally mix signals as shown in Equation 5. When data collection is triggered on the ROACH, data points are collected and stored in BRAM until the BRAM is full. When the BRAM is full, the number of data points taken is constricted. BRAM files can only store 2048 “32-bit signed” integers and because the sample rate is 200MHz, our samples time was about 10.24 μ s.

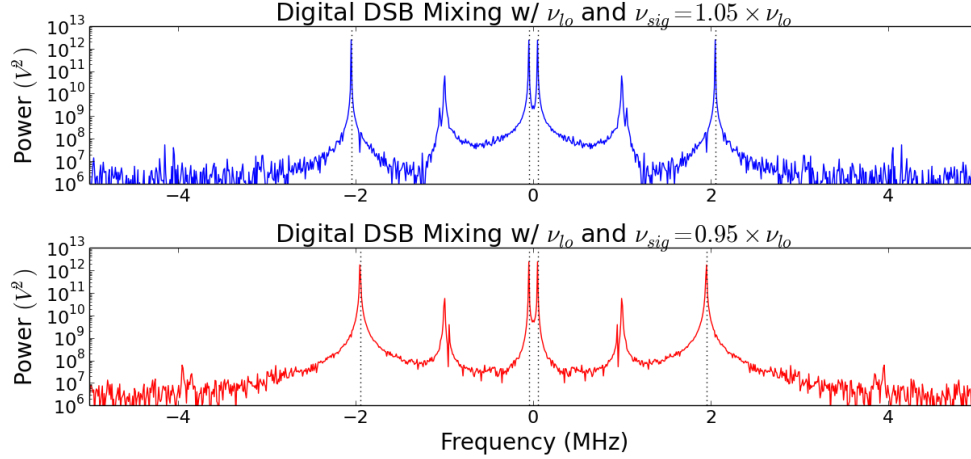


Figure 11: The graph above shows the power spectrum of the digital of the DSB mixing signal. The dotted lines show the 2.05MHz upper band on the first graph, 50kHz lower band, and 1.95MHz band on the second graph.

Although we wanted to be consistency with our measurements, because of problem with data collection, we used a local oscillator frequency of 1MHz along with a 1.05MHz and 0.95MHz sine wave. Additionally, the clock of the ROACH was set at a rate of 200MHz. The clock sets the rate at which logic gates in the FPGA execute operations and also defines the rate at which data points are sampled. The results of digitally mixing these signal frequencies can be seen in Figure 10. The waveform and power spectrum are very similar to the results of the analog mixing case, with the same spectral features.

However, the signal appears wider than the the DSB result of analog mixing using the Pulsar sampler (see Figure 7). The wider peaks are caused by the limitations of our sampling time. When samples are taken for longer periods of time, the frequency resolution of our spectrum increases. The spectrum of the digital mixing (see Figure 11) is also lot more smooth than the analog DSB mixing despite the DC offset. This is because the digital mixing product only contains the mixed frequency. In Figure 8, there are many noticeable higher-order frequency components. Additionally, there appear to be less frequencies in the spectrum of the digitally mixed DSB signal.

3.3.2 SSB Mixing

Although DSB mixing is useful, it can become problematic when mixing involves a “real” local oscillator signal. Single sideband (or SSB) mixing is one method of getting around this problem. Real signals require a superposition of the sine wave incorporating both of the positive and negative frequencies (see Figure 12). SSB mixing works by taking a signal and splitting it into two identical signals, one a cosine wave and another a sine wave. The cosine mixed signal is then treated as the real component of the mixed signal while the mixed sine wave represents the magnify components. The power spectrum of this mix is shown in Figure 13.

ν_{lo}	ν_{signal}	$\nu_{lo} \pm \nu_{signal}$
625kHz	1MHz	1.625MHz, -375kHz

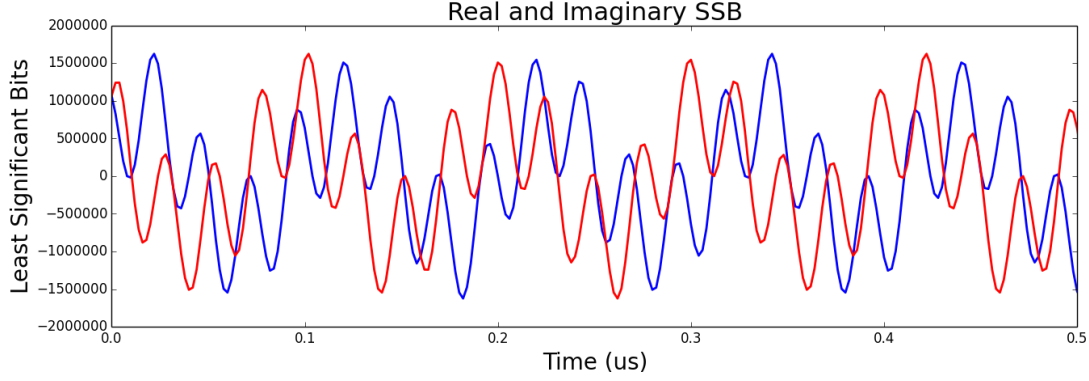


Figure 12: The graph above shows SSB digital mixing of a 10MHz complex signal with the blue function representing the real part and the red representing the imaginary. Data was collected under a sample rate of 200MHz.

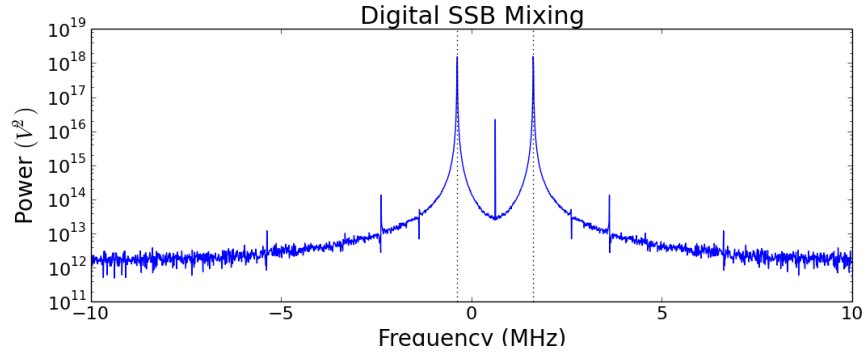


Figure 13: The graph above shows SSB digital mixing of a 10MHz complex signal with the blue function representing the real part and the red representing the imaginary. Data was collected under a sample rate of 200MHz.

In Figure 12, the positive frequency (cosine wave) is indicated by the real component of the filtered signal rising and falling slightly out of phase with the imaginary component (sine wave) of the signal. In both of the cosine and sine wave mixes, the frequency $v_{lo} - v_{signal}$ are the frequency of interest. Unlike DSB mixing, SSB mixing does not mix signals in a superposition of frequencies. Because the power spectrum is shifted with SSB mixing, concepts of “upper” and “lower” sidebands do not make sense. Since the input signal is real (see Equation 2), there must be two distinctive frequencies in the mixed product. Using a Fourier filter, the frequencies were isolated from from the mixed signals. The sign of the lower frequency, however, is determined from whether or not v_{lo} is larger than the v_{signal} .

3.4 FIR Filter Response

3.4.1 FIR Coefficients

The shape of the FIR filter chosen was determined by the desired bandpass my partners and I specified. The impulse response of this bandpass was set by the number of coefficients. The coefficients representing the impulse response is called “taps” (see Figure 3). The order of the filter we used was one less than the number of taps. As mentioned earlier, the FIR filter we used for in

the ROACH at our lab is an 8-tap FIR filter which we derived from writing the coefficient registers to the FPGA.

Coefficients		
Decimal Floating	Binary Fixed Point	Hex representation
0.125	0.0010000000000000	\x00\x00\x40\x00
-0.0517767	1.1111001010111101	\x00\x03\xe5\x7d
-0.125	1.1110000000000000	\x00\x03\xc0\x00
0.3017767	0.01001101010000010	\x00\x00\x9a\x83
0.625	0.1010000000000000	\x00\x01\x40\x00
0.3017767	0.01001101010000010	\x00\x00\x9a\x83
-0.125	1.1110000000000000	\x00\x03\xc0\x00
-0.0517767	1.1111001010111101	\x00\x03\xe5\x7d

Table 1: Coefficients used for the Digital Down Conversion on the Roach. Each number after the backslash (\) and x represent the value of 4 bits, so each two numbers holds a byte. We can see that summed they evaluate the 32 bits for the value of *coefficient*.

To choose coefficients for our FIR filter, we first designed our bandpass in the frequency domain and then apply the IFFT of the bandpass to retrieve our signal shape in the time domain. Since we did this digitally, we chose 8 points on the filter so that the IFFT would generate a complex coefficient for each of the eight registers. The filter we constructed for this lab is called a 5/8-filter. We call this a 5/8-filter because it was designed to filter the lower 5/8 portion of the frequencies that the FFT can represent. Additionally, because filter is designed to be symmetric about 0Hz, all of the filter's coefficients had to be real. Since our FPGA's clock or sample rate is 200MHz, the Nyquist frequency of our sampling is about 100 MHz. This made our desired frequency range from about -100MHz to 100MHz. The full response of our 5/8-band filter is 5/8 of this range; from -62.5 MHz to 62.5 MHz. The coefficients calculated for the 5/8 filter can be found in Table 1.

The program we used on the ROACH was set to handle 18-bit fixed point numbers. The numbers listed in the "Decimal Floating Point" column had to be converted to hex-digits representing each bit for the fixed point numbers. This is done by converting filter coefficients into an integer that have the same bits as the 18-bit fixed point representation. Using the bits contained in the integer, the hex-digits values were then used to control the ROACH. Using Python, my partners and I generated both hex digits used to control the ROACH along with the binary representations of the filter coefficients. Because there are only 8 coefficients, the coefficients alone do not provide a fine frequency resolution. To get the prediction of the response of our filter have a "finer" frequency resolution, my partners and I placed zeroes on the end of the list for the filter's coefficients, virtually increasing our sample time.

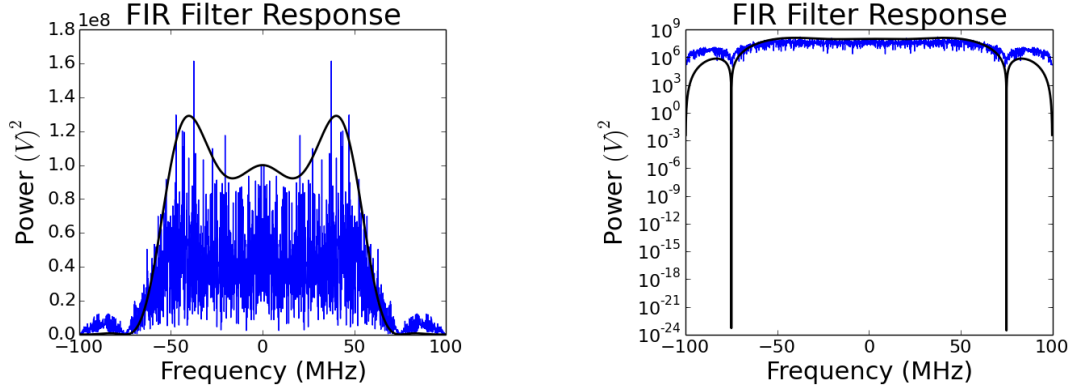


Figure 14: The graph above shows the FIR filter response using a -10 dBm, 100MHz frequency noise source input. The local oscillated frequency was set to 0Hz. The black line represents the filter's response.

3.4.2 Testing FIR Filter Response

After calculating our coefficients and producing a theoretical response curve of the filter, the actual response of the filter can be examined experimentally. By taking the noise 100MHz shown in Figure 14 and applying the filter to it, the outline of the power spectrum can be obtained.

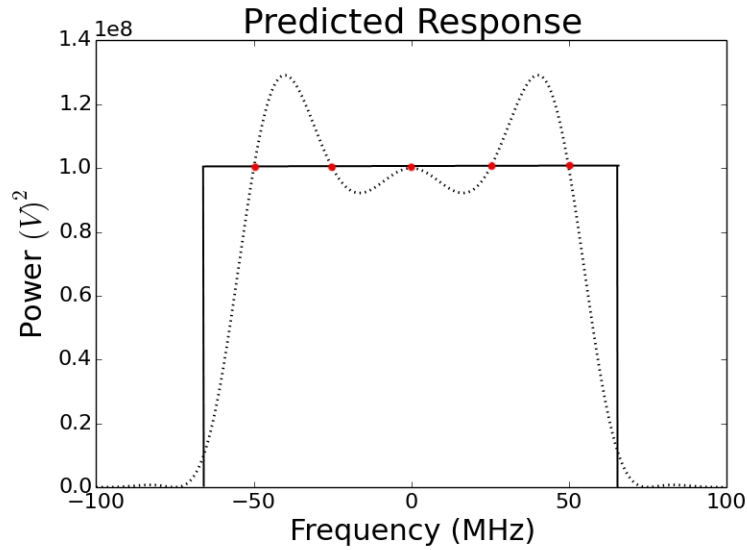


Figure 15: The graph above shows the FIR filter response. Dotted line shows predicted filter response, red points represent the coefficients, and solid black line represents the response of the filter.

Although this is not really a precise measurement, but it provides us with a rough estimate that the FIR filter we constructed is doing what it is supposed to be. By taking the ratio of the filtered power spectrum to the unfiltered power spectrum at its peak frequencies, a more precise response of the FIR filter can be obtained. What Figure 15 shows is ratio between the square of the filtered to the square of the unfiltered signal.

4 Conclusion

Signal processing is used in many applications and fields of study, especially Radio Astronomy. Through testing the Nyquist criterion, my partners and I found that frequencies up to one-half the sampling rate can be accurately facsimiliated. In this lab, we digitally sampled signals, analog and digitally mixed signals, performed various methods of filtration, and examined the FIR filter response set by the ROACH.

Additionally, my partners and I examined both analog and digital methods of DSB and SSB signal mixing. We were also able to make comparisons with the advantages and disadvantages of each. Through using techniques like Fourier filtering, digital filtration was made possible. Since digital mixing and signal filtration are used regularly in Radio Astronomy, understanding just what exactly these signals are and how they can be filtered is extremely relative.

In the end, I also learned about another method of digital filtering in the down conversion process using a FIR filter. FIR filters allow us to filter out unwanted frequencies from a DSB signal mixer. However, we were limited by the number of taps or coefficients used to convolve our signal. For purposes of isolating a certain frequency, the “rippled” shape of our the filter response (see Figure 15) will worked well since the response eliminated frequencies outside our bandwidth. By combining a FIR filter set with a specified sampling rate and using a heterodyne mixer, radio signals can be converted into useful forms for analysis.

5 References

1. Lab Partners: Leonardo Cassara and Caleb Levy.
2. Additional data was provided by Isaac Domagalski.
3. Help with data collection, analysis, and the report was also proved by Baylee Bordwell, Eduardo Herrera, and especially Isaac Domagalski.
4. Undergraduate radio lab. (2014, March 7). Retrieved from https://casper.berkeley.edu/astrobaki/index.php/Undergraduate_Radio_Lab.